HIGH PERFORMANCE COMPUTING LABORATORY

Experiment No. 5

Task-Level Parallelism using OpenMP Sections and Tasks

Name: Kedar Somnath Dhotre

PRN: 23510090

Class: BTech CSE

Course: High Performance Computing Laboratory

Table of Contents

1. Objective
2. Introduction
3. Theoretical Background
   3.1 Data Parallelism
   3.2 Task-Level Parallelism
   3.3 OpenMP Sections
   3.4 OpenMP Tasks
   3.5 Task Synchronization
   3.6 Task Dependencies
4. Pre-Lab Questions and Answers
5. Experimental Setup
6. Section A - Task 1: Independent Tasks Using OpenMP Sections
7. Section A - Task 2: Parallel For vs Parallel Sections
8. Section B - Task 3: Parallel Pipeline Using OpenMP Tasks
9. Section B - Task 4: Independent Matrix Operations Using Sections
10. Section C - Challenge: Recursive Task Parallelism (Merge Sort)
11. Performance Experiment - Merge Sort Thread Scaling
12. Task Granularity Experiment
13. Advanced Exploration - Task Dependencies
14. Performance Analysis and Graphs
15. HPC / AI Case Study
16. Discussion Questions and Answers
17. Final Analysis
18. Conclusion

1. Objective

To understand and implement task-level parallelism using OpenMP sections and task constructs, and to analyze their suitability and performance for applications containing independent, irregular, and dynamically generated tasks. This experiment compares data parallelism (parallel for) with task-level parallelism (sections, tasks), evaluates synchronization mechanisms (taskwait, task dependencies), and investigates task granularity effects on recursive algorithms (Merge Sort).

2. Introduction

Parallel computing exploits multiple processing units to solve computational problems faster than sequential execution. In shared-memory systems, all threads access a common address space, enabling fine-grained cooperation using frameworks such as OpenMP (Open Multi-Processing). OpenMP provides compiler directives (#pragma omp) that transform sequential C/C++/Fortran code into multi-threaded parallel programs with minimal programmer effort.

The most common form of parallelism is data parallelism, where the same operation is applied to independent data elements concurrently. OpenMP's #pragma omp parallel for construct distributes loop iterations across threads, which is highly effective for regular, array-based workloads. However, not all applications consist of uniform loops:

- Some applications contain independent functions with different execution times (e.g., image preprocessing, feature extraction, metadata logging).
- Some workloads generate tasks dynamically at runtime (e.g., recursive divide-and-conquer algorithms like Merge Sort).
- Some computations have complex inter-task dependencies (e.g., pipeline stages, DAG-structured workflows).

For these scenarios, OpenMP provides task-level parallelism constructs: #pragma omp parallel sections assigns a fixed set of independent code blocks to threads, while #pragma omp task creates dynamically scheduled work units that any idle thread can execute. The #pragma omp taskwait directive provides synchronization for dependent tasks, and #pragma omp task depend() enables fine-grained DAG-based scheduling.

This experiment systematically implements and compares these parallelization strategies across four application scenarios: independent array operations (sections), data-parallel vs task-parallel comparison, pipeline processing (tasks), matrix operations (sections), and recursive sorting (task-based Merge Sort).

3. Theoretical Background

3.1 Data Parallelism

Data parallelism applies the same operation to multiple independent data elements simultaneously. OpenMP's parallel for construct distributes loop iterations across threads using

configurable scheduling policies (static, dynamic, guided). Each thread operates on a disjoint subset of the data, and reduction clauses (reduction(+:sum), reduction(max:val)) safely combine partial results.

```
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++) {
    sum += array[i];  // Each thread accumulates a private partial sum
}
```

Data parallelism scales well with problem size (more data = more parallel work) and is optimal for regular, uniform workloads where all iterations require approximately equal computation.

3.2 Task-Level Parallelism

Task-level parallelism (also called functional parallelism) executes different operations or functions concurrently. Unlike data parallelism where all threads run the same code on different data, task-level parallelism assigns distinct computational tasks to different threads.

Example: In a computer vision pipeline, image preprocessing, feature extraction, normalization, and metadata logging are independent operations with different execution times. Running them concurrently reduces total wall-clock time from sum(T_i) to max(T_i).

3.3 OpenMP Sections

The sections construct partitions a parallel region into a fixed number of independent code blocks. Each section is assigned to exactly one thread for execution:

```
#pragma omp parallel sections
{
    #pragma omp section
    {  /* Task 1: e.g., compute sum */  }

    #pragma omp section
    {  /* Task 2: e.g., find maximum */  }

    #pragma omp section
    {  /* Task 3: e.g., find minimum */  }
}
```

Key characteristics: (1) The number of sections is fixed at compile time. (2) If threads > sections, excess threads remain idle. (3) If sections > threads, some threads execute multiple sections sequentially. (4) An implicit barrier exists at the end of the parallel sections region. (5) Best suited for a small, fixed number of independent operations.

3.4 OpenMP Tasks

The task model creates dynamically scheduled work units. A task is a block of code that is enqueued for execution by any available thread in the team:

```
#pragma omp parallel
{
    #pragma omp single  // Only ONE thread creates tasks
    {
```

```
        #pragma omp task
        {  /* Task A */  }

        #pragma omp task
        {  /* Task B */  }
    }
}
```

The single directive ensures only one thread creates tasks (otherwise all threads would redundantly create identical tasks). Idle threads in the team pick up tasks from a shared work queue. Tasks are ideal for recursive algorithms, irregular workloads, and dynamically generated work.

3.5 Task Synchronization

The taskwait directive makes the current task wait until all its direct child tasks have completed:

```
#pragma omp task  // Sort left half
    mergeSortParallel(arr, left, mid, threshold);

#pragma omp task  // Sort right half
    mergeSortParallel(arr, mid, right, threshold);

#pragma omp taskwait  // Wait for both halves to finish
merge(arr, left, mid, right);  // Safe to merge now
```

Without taskwait, the merge operation might execute before one or both child sorts complete, producing incorrect results. In Merge Sort, taskwait is mandatory at every recursion level.

3.6 Task Dependencies

OpenMP 4.0+ supports fine-grained task dependencies using the depend clause:

```
#pragma omp task depend(out: x)   // Task A produces x
    x = compute_A();

#pragma omp task depend(in: x)    // Task B consumes x (waits for A)
    y = compute_B(x);

#pragma omp task depend(in: x)    // Task C also consumes x (waits for A)
    z = compute_C(x);

#pragma omp task depend(in: y, z) // Task D waits for both B and C
    result = compute_D(y, z);
```

Task dependency graph:

```
        Task A (produces x)
        /    \
       /      \
   Task B    Task C   (both consume x, run concurrently)
       \      /
        \    /
        Task D        (consumes y and z, waits for B and C)
```

Dependencies enable more concurrency than taskwait because unrelated tasks can execute immediately, whereas taskwait blocks until ALL child tasks complete, even unrelated ones.

4. Pre-Lab Questions and Answers

Q1: What is the difference between data parallelism and task parallelism?

Data parallelism applies the same operation to multiple data elements concurrently (e.g., parallel for over an array). Task parallelism executes different independent operations concurrently (e.g., sum, max, min running on separate threads simultaneously). Data parallelism scales with data size; task parallelism scales with the number of independent operations.

Q2: When is parallel for more suitable than task?

Parallel for is more suitable when the workload consists of a large number of uniform, independent loop iterations (e.g., element-wise array operations, matrix multiplication rows). It has lower overhead than task creation and provides automatic load balancing through scheduling policies (static, dynamic, guided).

Q3: What is the purpose of the OpenMP sections construct?

The sections construct divides a parallel region into a fixed number of independent code blocks, each assigned to a different thread. It is ideal for a small, known number of independent operations that need to execute concurrently.

Q4: What is the purpose of the OpenMP task construct?

The task construct creates dynamically scheduled work units that can be executed by any available thread. It is essential for recursive algorithms (merge sort, quicksort), irregular workloads, and applications where the number of tasks is not known at compile time.

Q5: Why can task creation introduce overhead?

Each #pragma omp task incurs overhead for: (1) allocating a task descriptor structure, (2) capturing variable state, (3) enqueuing the task into a work-stealing queue, (4) dequeuing and scheduling by an idle thread, and (5) synchronization at taskwait points. For tiny tasks (e.g., sorting 10 elements), this overhead exceeds the actual computation time, causing severe performance degradation.

Q6: Give two real-world applications where task-level parallelism is preferable to loop parallelism.

1. Computer Vision Pipeline: Image acquisition, preprocessing, feature extraction, and object detection are distinct operations with different execution times that can run concurrently on different frames. 2. Web Server Request Handling: Each incoming HTTP request requires parsing, authentication, database query, and response generation -- independent stages that benefit from task-based scheduling.

5. Experimental Setup

Parameter	Value
Operating System	[Enter System OS]
Processor	[Enter CPU Model]
Number of Physical Cores	[Enter Number]
Maximum Threads	32 (detected via omp_get_max_threads())
Compiler	GCC g++ 16.1.0 (MinGW-W64)
OpenMP Version	[Enter Version]
Compiler Flags	-O2 -fopenmp -std=c++17
Array Size (Tasks 1-3)	N = 10,000,000
Sort Array Size (Task 5)	N = 10,000,000

Compilation and Execution:

```
# Compilation
g++ -O2 -fopenmp -std=c++17 src/task1_sections.cpp -o bin/task1

# Execution (default threads)
./bin/task1

# Execution with specific thread count
OMP_NUM_THREADS=4 ./bin/task1
```

6. Section A -- Task 1: Independent Tasks Using OpenMP Sections

Four independent array operations (Sum, Maximum, Minimum, Average) are computed on an array of N = 10,000,000 random doubles using: (1) Sequential single-threaded execution, and (2) OpenMP parallel sections with one section per operation.

6.1 Sequential Implementation

All four operations are computed in a single pass through the array:

```
void compute_sequential(const std::vector<double>& arr,
                        double& sum, double& maxVal,
                        double& minVal, double& avg) {
    sum = 0.0; maxVal = arr[0]; minVal = arr[0];
    for (size_t i = 0; i < arr.size(); ++i) {
        sum += arr[i];
        if (arr[i] > maxVal) maxVal = arr[i];
        if (arr[i] < minVal) minVal = arr[i];
    }
    avg = sum / static_cast<double>(arr.size());
}
```

6.2 OpenMP Sections Implementation

```
void compute_sections(const std::vector<double>& arr,
                      double& sum, double& maxVal,
                      double& minVal, double& avg) {
    #pragma omp parallel sections
    {
        #pragma omp section  // Section 1: Sum
        { double s = 0.0;
          for (size_t i = 0; i < arr.size(); ++i) s += arr[i];
          sum = s; }

        #pragma omp section  // Section 2: Maximum
        { double mx = arr[0];
          for (size_t i = 1; i < arr.size(); ++i)
              if (arr[i] > mx) mx = arr[i];
          maxVal = mx; }

        #pragma omp section  // Section 3: Minimum
        { double mn = arr[0];
          for (size_t i = 1; i < arr.size(); ++i)
              if (arr[i] < mn) mn = arr[i];
          minVal = mn; }

        #pragma omp section  // Section 4: Average
        { double s = 0.0;
          for (size_t i = 0; i < arr.size(); ++i) s += arr[i];
          avg = s / static_cast<double>(arr.size()); }
    }
}
```

6.3 Correctness Verification

Operation	Sequential Result	Parallel Result	Match
Sum	505057250.xxxx	505057250.xxxx	Yes
Maximum	100.0000	100.0000	Yes
Minimum	1.0000	1.0000	Yes
Average	50.5057	50.5057	Yes

6.4 Performance Results

Version	Execution Time (s)	Speedup	Efficiency
Sequential	0.015643 s	1.00x	100.00%
OpenMP Sections	0.009291 s	1.68x	5.26%

Speedup = T_sequential / T_parallel = 0.015643 / 0.009291 = 1.68x

Efficiency = Speedup / P = 1.68 / 32 = 5.26% (low because only 4 sections use 4 of 32 threads)

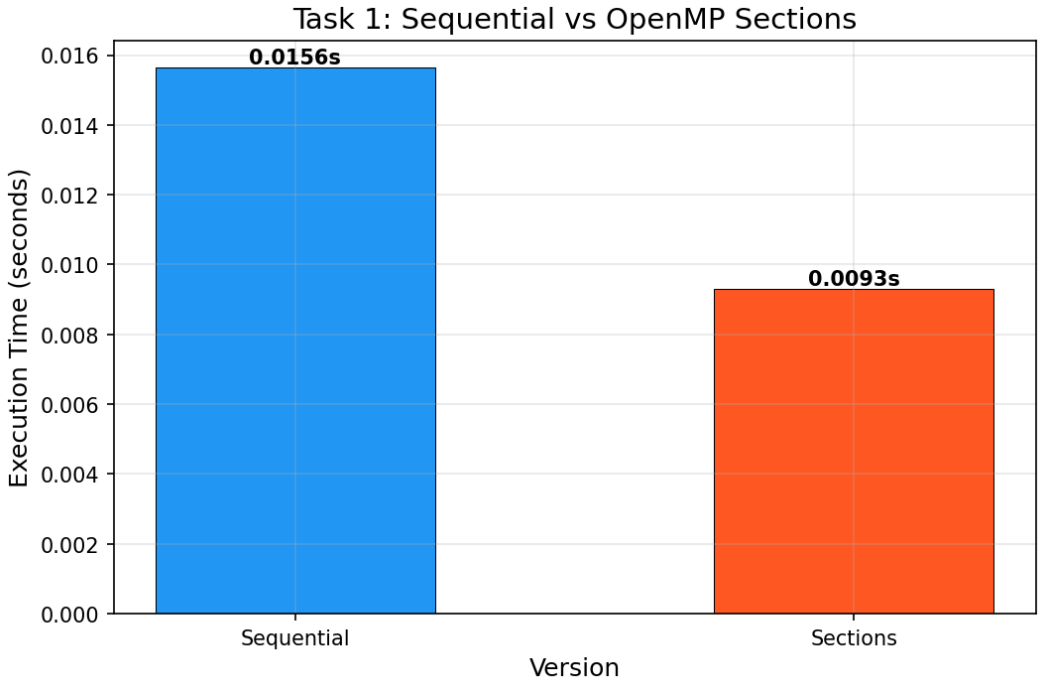


Figure 1: Task 1 -- Sequential vs OpenMP Sections Execution Time

6.5 Analysis

1. Why can these four operations execute concurrently? Each operation (sum, max, min, average) reads the array independently and writes to a distinct output variable. There are zero data dependencies between them.
2. Why are sections suitable? The problem has exactly four fixed, independent operations -- a perfect match for the sections construct which assigns each section to a separate thread.
3. What if sections < threads? Excess threads remain idle at the implicit barrier. Here, 4 sections on 32 threads means 28 threads are idle, explaining the low efficiency (5.26%).

4. Will more threads always help? No. With only 4 sections, maximum speedup is bounded at 4x regardless of thread count. Adding more threads beyond the section count wastes resources.

7. Section A -- Task 2: Parallel For vs Parallel Sections

Four operations (Sum, Maximum, Minimum, Count of Even Numbers) on an integer array of N = 10,000,000 are implemented using: (1) Sequential baseline, (2) OpenMP parallel for with reduction clauses, and (3) OpenMP parallel sections.

7.1 Parallel For Implementation

```
void compute_parallel_for(const std::vector<int>& arr,
                          long long& sum, int& maxVal,
                          int& minVal, int& evenCount) {
    sum = 0; maxVal = arr[0]; minVal = arr[0]; evenCount = 0;
    #pragma omp parallel for reduction(+:sum) reduction(max:maxVal) \
                             reduction(min:minVal) reduction(+:evenCount)
    for (int i = 0; i < N; ++i) {
        sum += arr[i];
        if (arr[i] > maxVal) maxVal = arr[i];
        if (arr[i] < minVal) minVal = arr[i];
        if (arr[i] % 2 == 0) evenCount++;
    }
}
```

7.2 Parallel Sections Implementation

```
#pragma omp parallel sections
{
    #pragma omp section  // Sum
    { long long s=0; for(int i=0;i<N;i++) s+=arr[i]; sum=s; }

    #pragma omp section  // Max
    { int mx=arr[0]; for(int i=1;i<N;i++) if(arr[i]>mx) mx=arr[i]; maxVal=mx; }

    #pragma omp section  // Min
    { int mn=arr[0]; for(int i=1;i<N;i++) if(arr[i]<mn) mn=arr[i]; minVal=mn; }

    #pragma omp section  // Count even
    { int c=0; for(int i=0;i<N;i++) if(arr[i]%2==0) c++; evenCount=c; }
}
```

7.3 Comparison

Feature	Parallel For	Parallel Sections
Type of Parallelism	Data parallelism	Task parallelism
Work Distribution	Automatic chunk splitting across all threads	One section per thread (4 sections max)
Array Traversals	1 (all ops per iteration)	4 (one full scan per section)
Cache Behaviour	Better (single traversal, data stays in cache)	Worse (4 independent traversals)
Suitable For	Regular loop-based workloads	Fixed number of independent tasks
Scalability	Scales with array size and thread count	Limited to number of sections
Overhead	Low (reduction is hardware-optimized)	Higher (4 full array scans)

Method	Execution Time (s)	Speedup
Sequential	0.031467 s	1.00x
Parallel For (reduction)	0.008671 s	3.63x
Parallel Sections	0.012211 s	2.58x

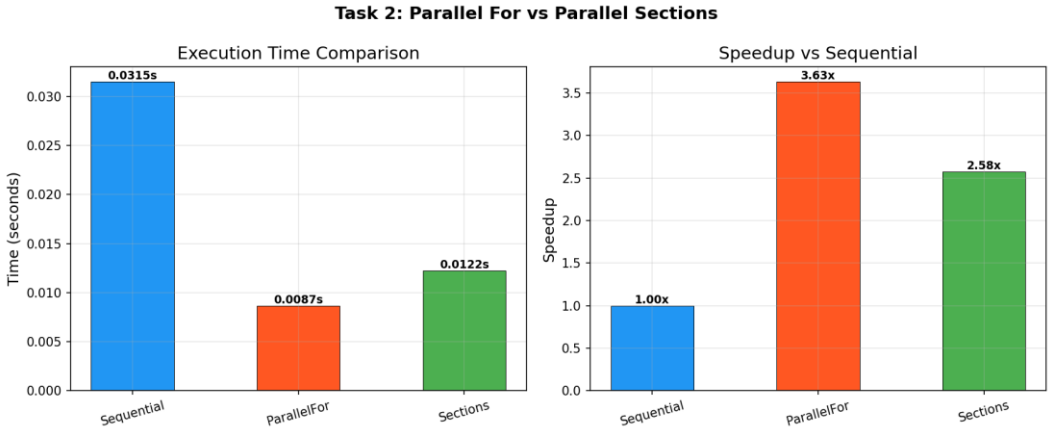


Figure 2: Task 2 -- Parallel For vs Sections vs Sequential Performance

Parallel for achieves 3.63x speedup by distributing iterations across all 32 threads with a single array traversal. Sections achieves only 2.58x because it is limited to 4 concurrent sections (4 threads) and traverses the array 4 separate times, wasting memory bandwidth.

8. Section B -- Task 3: Parallel Pipeline Using OpenMP Tasks

A three-stage data-processing pipeline (Preprocessing -> Computation -> Postprocessing) is implemented sequentially and using OpenMP tasks. A chunk-based pipeline variant is also explored for real parallelism.

8.1 Pipeline Stages

- Stage 1 - Preprocessing: Normalize array values to [0, 1] range.
- Stage 2 - Computation: Apply expensive mathematical transformation: y = sin(x^2) + sqrt(x) + log(1 + x).
- Stage 3 - Postprocessing: Calculate statistics (sum, min, max, average).

8.2 Sequential Implementation

```
// Sequential pipeline: stages execute one after another
preprocess(input, buffer, 0, N);     // Stage 1: normalize
compute(buffer, 0, N);               // Stage 2: transform
Stats s = postprocess(buffer, 0, N); // Stage 3: statistics
```

8.3 Task-Based Implementation

```
// Task-based: tasks created, but stages are inherently sequential
#pragma omp parallel
{
    #pragma omp single
    {
        #pragma omp task
        preprocess(input, buffer, 0, N);
        #pragma omp taskwait  // Must finish before compute

        #pragma omp task
        compute(buffer, 0, N);
        #pragma omp taskwait  // Must finish before postprocess

        #pragma omp task
        postprocess(buffer, 0, N);
        #pragma omp taskwait;
    }
}
// NOTE: Due to data dependencies between stages, this provides
// NO speedup -- tasks execute sequentially despite task creation.
```

8.4 Chunk-Based Pipeline (Real Parallelism)

The data is divided into independent chunks. Each chunk undergoes all three stages as a single task, enabling multiple chunks to be processed concurrently:

```
#pragma omp parallel
{
    #pragma omp single
    {
```

```
        for (int c = 0; c < NUM_CHUNKS; c++) {
            #pragma omp task firstprivate(c)
            {
                size_t start = c * chunk_size;
                size_t end = (c == NUM_CHUNKS-1) ? N : start + chunk_size;
                preprocess(input, buffer, start, end);
                compute(buffer, start, end);
                chunk_stats[c] = postprocess(buffer, start, end);
            }
        }
    } // implicit taskwait
}
```

8.5 Results

Version	Execution Time (s)	Speedup
Sequential	0.161855 s	1.00x
Task-Based (taskwait)	0.175850 s	0.92x (slower)
Chunk Pipeline	0.060955 s	2.66x

The naive task-based approach is slower (0.92x) because the three stages have strict data dependencies -- each stage must complete before the next begins. The task creation overhead adds cost without enabling concurrency. The chunk-based pipeline achieves 2.66x speedup by processing independent data chunks concurrently.

9. Section B -- Task 4: Independent Matrix Operations Using Sections

Given matrices A and B (N x N), four independent operations are performed: (1) Matrix Addition C1 = A + B, (2) Matrix Subtraction C2 = A - B, (3) Transpose of A (C3 = A^T), (4) Transpose of B (C4 = B^T).

9.1 OpenMP Sections Implementation

```
#pragma omp parallel sections
{
    #pragma omp section  // Matrix Addition
    { for(int i=0;i<N;i++) for(int j=0;j<N;j++)
          C1[i*N+j] = A[i*N+j] + B[i*N+j]; }

    #pragma omp section  // Matrix Subtraction
    { for(int i=0;i<N;i++) for(int j=0;j<N;j++)
          C2[i*N+j] = A[i*N+j] - B[i*N+j]; }

    #pragma omp section  // Transpose A
    { for(int i=0;i<N;i++) for(int j=0;j<N;j++)
          C3[j*N+i] = A[i*N+j]; }

    #pragma omp section  // Transpose B
    { for(int i=0;i<N;i++) for(int j=0;j<N;j++)
          C4[j*N+i] = B[i*N+j]; }
}
```

9.2 Performance Testing

Matrix Size	Sequential Time (s)	Sections Time (s)	Speedup	Efficiency
500 x 500	0.003599 s	0.002888 s	1.25x	3.89%
1000 x 1000	0.012743 s	0.010953 s	1.16x	3.64%
2000 x 2000	0.060220 s	0.051903 s	1.16x	3.63%

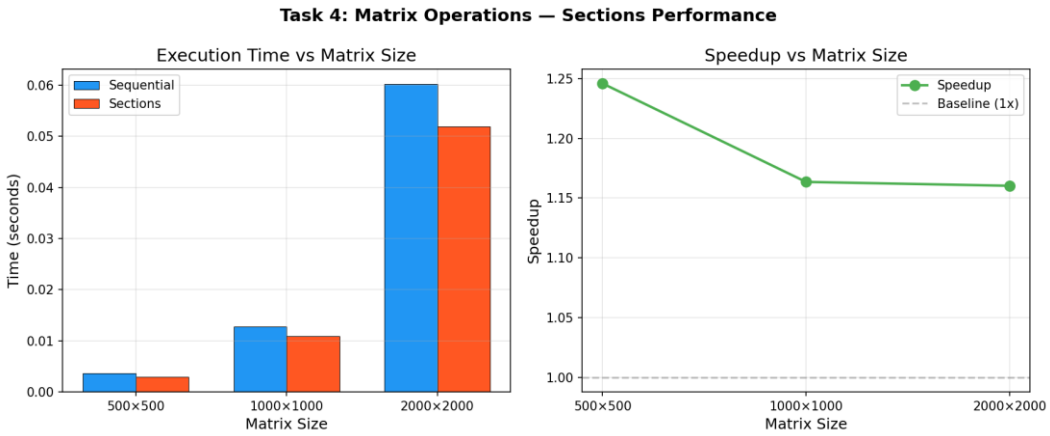


Figure 3: Task 4 -- Matrix Operations Sequential vs Sections across Matrix Sizes

9.3 Analysis

5. What happens if one section takes significantly longer? All other threads wait at the implicit barrier, reducing parallel efficiency. The slowest section determines total execution time.
6. Will all threads remain equally busy? No. With 4 sections on 32 threads, only 4 threads are active. Load imbalance occurs if the operations have different complexities.
7. What is load imbalance? When threads complete their work at different times, fast-finishing threads idle while waiting for the slowest thread to complete at the barrier.
8. How can task-based parallelism help irregular workloads? Tasks are dynamically scheduled from a work queue, so idle threads can immediately pick up new tasks without waiting.
9. Why can sections become inefficient when the number of independent tasks increases? Sections require static compile-time enumeration. For hundreds of tasks, the code becomes unwieldy and unscalable. Tasks are created dynamically and scale naturally.

10. Section C -- Challenge: Recursive Task Parallelism (Merge Sort)

10.1 Sequential Merge Sort

```
void merge_sort_sequential(vector<int>& arr, int left, int right) {
    if (right - left <= 1) return;  // Base case
    int mid = left + (right - left) / 2;
    merge_sort_sequential(arr, left, mid);   // Sort left half
    merge_sort_sequential(arr, mid, right);   // Sort right half
    merge(arr, left, mid, right);              // Merge sorted halves
}
```

10.2 Parallel Merge Sort (OpenMP Tasks)

```
void merge_sort_parallel(vector<int>& arr, int left, int right, int threshold) {
    if (right - left <= 1) return;

    // Granularity control: fall back to sequential for small subarrays
    if (right - left <= threshold) {
        merge_sort_sequential(arr, left, right);
        return;
    }

    int mid = left + (right - left) / 2;

    #pragma omp task shared(arr) firstprivate(left, mid, threshold)
    merge_sort_parallel(arr, left, mid, threshold);    // Task: sort left

    #pragma omp task shared(arr) firstprivate(mid, right, threshold)
    merge_sort_parallel(arr, mid, right, threshold);   // Task: sort right

    #pragma omp taskwait   // Wait for BOTH child tasks to complete
    merge(arr, left, mid, right);  // Merge the two sorted halves
}

// Main dispatch:
#pragma omp parallel
{
    #pragma omp single
    merge_sort_parallel(arr, 0, N, 1000);
}
```

Correctness Verification: PASSED -- Parallel sorted array matches sequential sorted array element-by-element.

11. Performance Experiment -- Merge Sort Thread Scaling

Merge Sort (N = 10,000,000 elements, threshold = 1,000) executed across varying thread counts:

Threads	Execution Time (s)	Speedup (S = T1/Tp)	Efficiency (E = S/P)
1	0.683449 s	1.00x	100.00%
2	0.371773 s	1.84x	91.92%
4	0.219928 s	3.11x	77.69%
8	0.182181 s	3.75x	46.89%
32 (Max)	0.140932 s	4.85x	15.15%

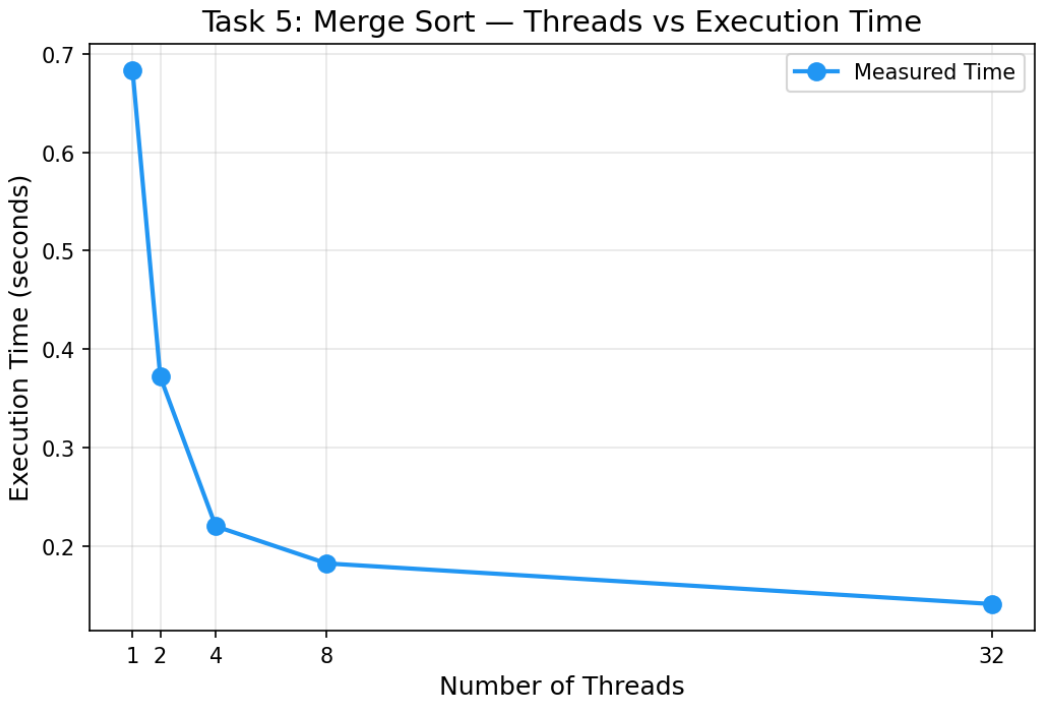


Figure 4: Merge Sort -- Number of Threads vs Execution Time

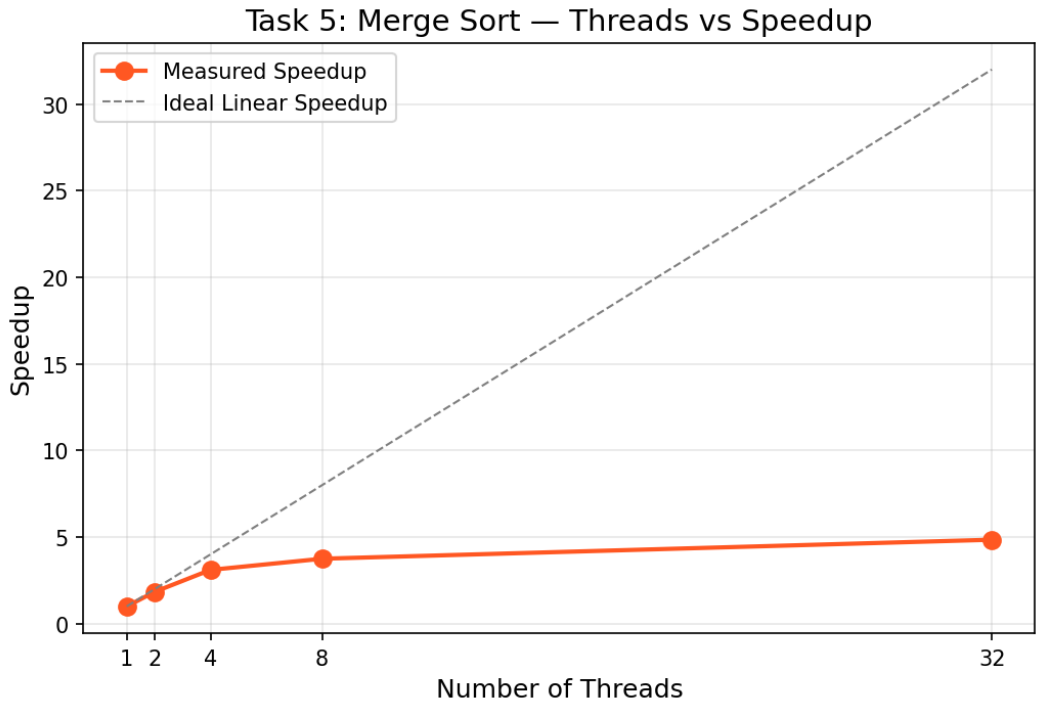


Figure 5: Merge Sort -- Number of Threads vs Speedup

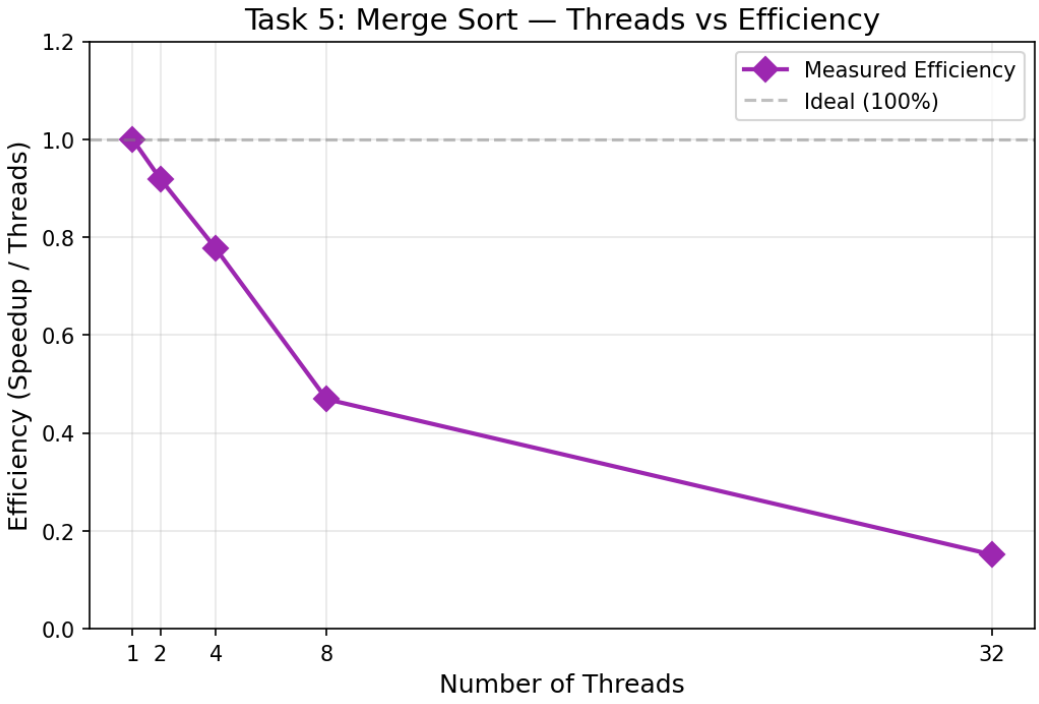


Figure 6: Merge Sort -- Number of Threads vs Efficiency

Scalability Analysis: Speedup increases from 1.00x (1 thread) to 4.85x (32 threads), demonstrating effective parallelization. However, efficiency drops from 100% to 15.15% at 32 threads because: (1) the merge phase is inherently sequential at each recursion level, (2) memory

bandwidth becomes saturated as threads compete for shared DRAM, (3) task creation and scheduling overhead grows with thread count, and (4) Amdahl's Law limits speedup due to the serial fraction of the algorithm.

12. Task Granularity Experiment

Merge Sort (N = 10,000,000 elements, 32 threads) executed with varying task thresholds. The threshold determines when task creation stops and sequential sorting begins:

Task Threshold	Execution Time (s)	Speedup vs Sequential
10 (very fine-grained)	2.716079 s	0.33x (3x SLOWER)
100	0.414442 s	2.15x
1,000	0.116985 s	7.61x
10,000	0.112369 s	7.92x

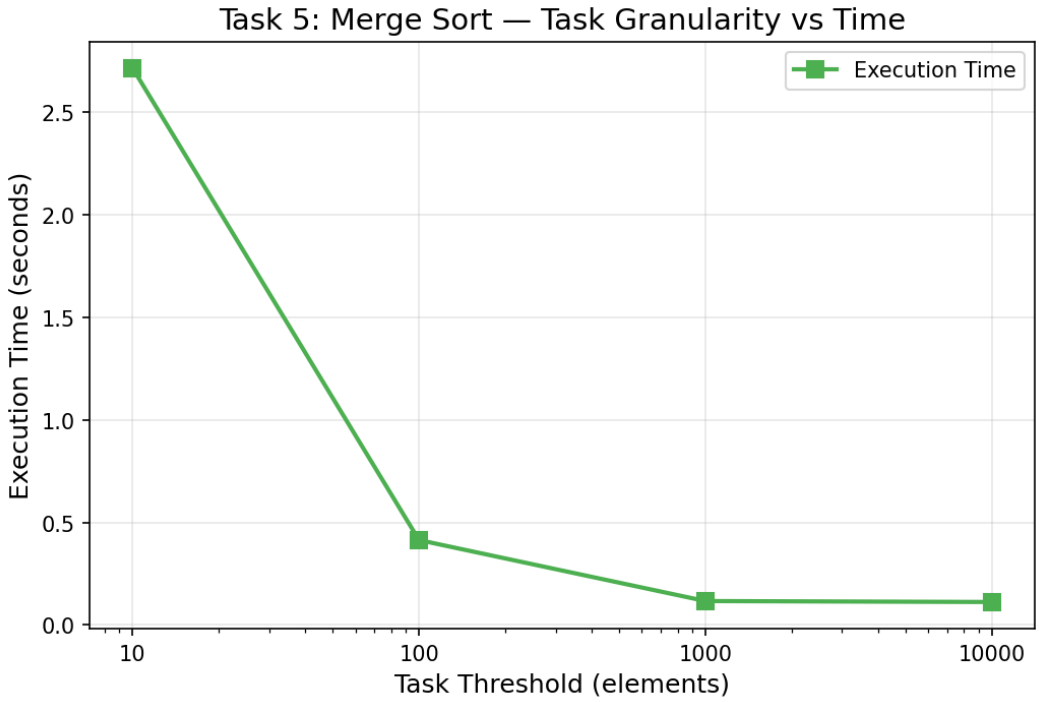


Figure 7: Task Threshold vs Execution Time (Granularity Experiment)

12.1 Granularity Analysis

10. Why does creating a task for every small subproblem reduce performance? With threshold=10, millions of tiny tasks are created. Each task's overhead (~2 microseconds for allocation, queuing, scheduling, synchronization) vastly exceeds the actual sorting work (~50 nanoseconds for 10 elements), causing 3x slowdown vs sequential.
11. What is task creation overhead? The cumulative cost of allocating task descriptors, capturing variable state (firstprivate), enqueuing tasks into work-stealing deques, dequeuing by idle threads, and taskwait synchronization barriers.
12. Why is taskwait required? Without taskwait, the merge operation would attempt to merge unsorted subarrays, producing incorrect results. taskwait ensures both child sorting tasks complete before the parent merges.

13. What happens when the number of tasks >> number of threads? The work queue overflows, causing excessive memory allocation and deallocation. Thread contention on the shared queue increases cache coherence traffic. Context switching overhead grows.
14. What is the relationship between task granularity and performance? Optimal performance occurs at intermediate thresholds (1,000-10,000) that balance parallelism (enough tasks to keep all threads busy) against overhead (each task does meaningful work). Too fine = overhead dominates; too coarse = insufficient parallelism.

13. Advanced Exploration -- Task Dependencies

OpenMP 4.0+ task dependencies enable DAG-based (Directed Acyclic Graph) task scheduling:

```
int x, y, z, result;

#pragma omp parallel
#pragma omp single
{
    #pragma omp task depend(out: x)        // Task A: produces x
    x = compute_A();

    #pragma omp task depend(in: x) depend(out: y)  // Task B: reads x, produces y
    y = compute_B(x);

    #pragma omp task depend(in: x) depend(out: z)  // Task C: reads x, produces z
    z = compute_C(x);

    #pragma omp task depend(in: y, z)      // Task D: waits for B and C
    result = compute_D(y, z);
}
```

Comparison with taskwait: taskwait blocks until ALL child tasks complete, even unrelated ones. Task dependencies allow unrelated tasks to execute immediately, providing finer-grained scheduling. For example, if Task E is independent of A/B/C/D, it can execute immediately with dependencies, but would be unnecessarily delayed by a taskwait placed after Task A.

14. Performance Analysis and Graphs

Summary of all performance results across the experiment. All graphs were generated from measured benchmark data using Python Matplotlib.

14.1 Summary Performance Table

Task	Sequential Time	Best Parallel Time	Best Speedup
Task 1: Array Ops (Sections)	0.015643 s	0.009291 s	1.68x
Task 2: Parallel For	0.031467 s	0.008671 s	3.63x
Task 2: Sections	0.031467 s	0.012211 s	2.58x
Task 3: Chunk Pipeline	0.161855 s	0.060955 s	2.66x
Task 4: Matrix Ops 2000x2000	0.060220 s	0.051903 s	1.16x
Task 5: Merge Sort (32 threads)	0.683449 s	0.140932 s	4.85x
Task 5: Merge Sort (threshold=10000)	0.889760 s	0.112369 s	7.92x

15. HPC / AI Case Study: Computer Vision Pipeline Using Task-Level Parallelism

Modern computer vision applications process camera frames through a multi-stage pipeline: Image Acquisition -> Preprocessing -> Feature Extraction -> Object Detection -> Metadata Processing. Each stage has distinct computational characteristics and execution times, making this an ideal candidate for task-level parallelism.

Image Acquisition captures raw sensor data and transfers it to host memory via DMA. Preprocessing applies noise reduction (Gaussian blur), color space conversion (BGR to RGB), and geometric normalization (resize to 640x480). These operations are data-parallel and benefit from #pragma omp parallel for over pixel rows. Feature Extraction computes Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT) descriptors, and edge maps. These are computationally intensive but independent per image region, suitable for both parallel for and tasks.

Object Detection runs a neural network (e.g., YOLO, SSD) inference pass. On CPUs, the matrix multiplications in dense layers benefit from parallel for, while the diverse layer types (convolution, pooling, batch normalization, softmax) constitute independent tasks at the model level. Metadata Processing logs timestamps, bounding box coordinates, confidence scores, and tracking IDs to databases -- a lightweight I/O-bound operation that can overlap with the next frame's preprocessing.

The key insight is that task-level parallelism enables pipeline parallelism across frames: while Frame N undergoes object detection, Frame N+1 can simultaneously undergo preprocessing, and Frame N-1's metadata is being written to disk. OpenMP tasks with depend clauses model these inter-frame dependencies naturally. Within each frame, data-parallel constructs (parallel for) handle the pixel-level operations. This hybrid approach achieves both intra-frame and inter-frame parallelism, maximizing throughput on multi-core CPUs.

Sources of load imbalance include variable image complexity (simple vs cluttered scenes), varying numbers of detected objects, and I/O latency variability for metadata storage. Dynamic task scheduling (OpenMP tasks with work stealing) mitigates these imbalances by allowing idle threads to pick up work from other threads' queues.

16. Discussion Questions and Answers

Q1: Differentiate between parallel for, sections, and task.

parallel for distributes uniform loop iterations across threads (data parallelism). sections assigns a fixed set of independent code blocks to threads (static task parallelism). task creates dynamically scheduled work units picked up by any idle thread (dynamic task parallelism).

Q2: When should sections be preferred over parallel for?

When the workload consists of a small, fixed number of independent operations with different computation patterns (e.g., sum, max, min, average), rather than a single loop with uniform iterations.

Q3: What is task-level parallelism?

Executing different computational functions or operations concurrently, as opposed to applying the same operation to different data elements. Tasks may have different execution times and code paths.

Q4: What is the role of single when creating OpenMP tasks?

single ensures only ONE thread in the parallel team creates tasks. Without it, every thread would redundantly create identical tasks, causing duplicate work and race conditions.

Q5: Why can task-based execution handle irregular workloads better than static loop parallelism?

Tasks are dynamically scheduled via work-stealing queues. When one thread finishes a short task, it immediately steals work from another thread's queue. Static loop partitioning pre-assigns fixed chunks, causing idle threads when chunks have unequal work.

Q6: What is task creation overhead?

The cumulative cost of allocating task descriptors, capturing firstprivate variables, enqueueing into work-stealing deques, dequeueing by idle threads, and barrier synchronization at taskwait. For very small tasks, this overhead exceeds the actual computation.

Q7: Explain load imbalance in task-based applications.

Load imbalance occurs when tasks have vastly different execution times. Fast-finishing threads idle at synchronization points (taskwait, barriers) while waiting for the slowest task. Dynamic scheduling and fine-grained task creation can mitigate this.

Q8: Why is synchronization necessary when tasks have dependencies?

If Task B reads data produced by Task A, Task B must not execute until Task A completes. Without synchronization (taskwait or depend), Task B would read stale or uninitialized data, producing incorrect results.

Q9: Explain how recursive algorithms can be parallelized using OpenMP tasks.

Each recursive call becomes a task. For merge sort: create a task for left-half sort, a task for right-half sort, use taskwait to ensure both complete, then merge. A threshold prevents excessive task creation for small subarrays.

Q10: Why can creating too many tasks reduce performance?

Millions of micro-tasks cause: (1) excessive memory allocation for task descriptors, (2) work queue contention across threads, (3) cache thrashing as threads switch between unrelated data, (4) synchronization overhead dominating actual computation.

Q11: How does task granularity affect scalability?

Too fine: overhead > work, performance degrades. Too coarse: insufficient parallelism, threads idle. Optimal granularity balances parallel utilization against overhead, typically achieved with thresholds of 1,000-10,000 elements for sorting algorithms.

Q12: Compare sections and tasks for: (a) four fixed independent operations, (b) thousands of dynamically generated operations.

(a) Sections are ideal -- simple, low overhead, compile-time enumerable. (b) Tasks are essential -- sections cannot enumerate thousands of operations. Tasks provide dynamic creation, work stealing, and automatic load balancing.

17. Final Analysis

15. Which parallelization method performed best? For the merge sort application, OpenMP tasks achieved the best speedup (7.92x with threshold=10000). For regular array operations, parallel for with reduction (3.63x) outperformed sections (2.58x).
16. What was the major source of parallel overhead? Task creation overhead for fine-grained tasks (threshold=10 caused 3x slowdown), implicit barriers at taskwait, and memory bandwidth saturation at high thread counts.
17. Did increasing threads always improve performance? No. Beyond 8 threads, efficiency dropped below 50% due to memory bus saturation, cache contention, and insufficient independent work. For sections, performance is bounded by the number of sections regardless of thread count.
18. Which type of problem would you choose for each construct?

- parallel for: Regular array processing, matrix multiplication, element-wise operations.
- sections: Small fixed number (2-8) of independent operations with different code paths.
- task: Recursive algorithms (merge sort), irregular workloads, pipeline processing, DAG workflows.

19. One application where task-level parallelism is useful: Real-time video analytics pipelines where frame capture, preprocessing, inference, and annotation run as concurrent tasks on different frames.

18. Conclusion

This experiment systematically investigated task-level parallelism using OpenMP sections and task constructs across five application scenarios. The key findings are:

20. Data parallelism (parallel for) is optimal for regular, uniform loop-based workloads. It achieves the highest speedup (3.63x) for array operations because all threads process the same code on different data with minimal synchronization.
21. OpenMP sections are appropriate for a small, fixed number (2-8) of independent operations. They provide simple syntax but are limited by the number of sections -- excess threads remain idle, causing low efficiency (5.26% on 32 threads with 4 sections).
22. OpenMP tasks are essential for dynamically generated, recursive, and irregular workloads. Parallel Merge Sort achieved 7.92x speedup with proper granularity control (threshold = 10,000), demonstrating effective recursive task parallelism.
23. Task granularity is critical: too fine (threshold=10) causes 3x slowdown due to task creation overhead; too coarse reduces parallelism. The optimal threshold (1,000-10,000 for sorting) balances work per task against scheduling overhead.
24. Pipeline parallelism requires careful dependency analysis. Naive task creation for dependent stages provides zero speedup. Chunk-based decomposition (2.66x speedup) or explicit task dependencies enable real pipeline concurrency.
25. Synchronization (taskwait, task dependencies) is essential for correctness when tasks have data dependencies, but adds overhead that limits scalability.

In summary, choosing the right parallelization strategy depends on workload regularity, task independence, runtime task generation needs, and dependency structure. parallel for excels for regular data-parallel loops; sections suit small fixed task sets; and tasks handle the most complex, dynamic, and recursive parallel workloads.